Few-Shot Bearing Fault Diagnosis: Practical MAML Reproduction

Reproduction report | 7 October 2026 | PyTorch | Real CWRU recordings | Reduced CPU budget

1. Abstract

This project implements the central method of Zhang et al. using an independently written four-block convolutional classifier and full second-order model-agnostic meta-learning (MAML). Real drive-end and fan-end vibration signals from the CWRU Bearing Data Center form paired 4,096-value inputs. The experiments evaluate adaptation to fault classes excluded from meta-training using five labeled support examples per class. The learned initialization and layer-specific inner learning rates are trained separately for three random seeds per task setting. Measured accuracy is 3-way 5-shot: 94.05 +/- 1.20%; 5-way 5-shot: 58.03 +/- 5.30%. The deviations are sample standard deviations across training seeds, not confidence intervals. The implementation, runnable notebook, checkpoints, per-query predictions and figures are supplied. The computation schedule and several underspecified preprocessing choices differ from the article; therefore, these measurements demonstrate a practical reproduction of the learning mechanism rather than an exact replication of the published benchmark.

2. Introduction

Bearings are central to rotating machinery. Defects in the rolling elements or raceways change the vibration response of the assembly. A supervised classifier ordinarily requires many labeled examples for each condition, including operating loads and defect severities that may be expensive to reproduce. A few-shot classifier instead receives a small labeled support set for a new classification task. The relevant question is whether knowledge learned from other fault classes helps it adapt quickly.

Here, N-way K-shot means N classes in an episode and K labeled support examples for each class. Query examples are different windows from those same classes and are used to measure adaptation. At meta-test time, these classes were absent from the initialization training. This distinction is essential: randomly dividing windows from every class into ordinary training and testing sets would measure a different problem. Both requested experiments are therefore constructed with disjoint class sets.

3. Original Paper

The reference is Shen Zhang, Fei Ye, Bingnan Wang and Thomas G. Habetler, Few-Shot Bearing Fault Diagnosis Based on Model-Agnostic Meta-Learning, arXiv:2007.12851v4 (2021) [1]. The implementation uses its CWRU class definitions, paired signal input and convolutional backbone. It implements both fixed and learnable inner learning rates; the reported experiments use the learnable variant. The article also studies Paderborn artificial-to-real transfer. That additional dataset, optimizer sweeps and the full set of fault splits are outside this submission. Table V is the reference for the two requested CWRU comparisons. No external implementation was copied into this project.

4. Dataset

Thirty original MATLAB files were downloaded from the official university pages [2-4]. The data cover ten classes at 1, 2 and 3 HP; load is not itself the class label. Fault sizes are 0.007, 0.014 and 0.021 inches. Outer-race records use the centered 6-o'clock position. The table below is computed from the loaded recordings after preprocessing, not copied from the article.

Class	Condition	Files	Windows
1	Healthy	3	177
2	Ball_007	3	177
3	Ball_014	3	177
4	Ball_021	3	177
5	IR_007	3	178
6	IR_014	3	177
7	IR_021	3	177
8	OR_007	3	177
9	OR_014	3	177
10	OR_021	3	177

The loader produces 1,771 paired, non-overlapping windows in total. A source manifest stores the exact channel keys, original lengths, assumed rates and SHA-256 hash of each file. A window manifest records recording ID, class, load and sample boundaries. A separate manifest for each run records the nine chosen training examples and the test support/query pools. Together with seeds and saved configurations, these records make the experiments auditable.

Healthy files 98, 99 and 100 are treated as 48 kHz recordings and both channels are decimated to 12 kHz using a polyphase anti-aliasing filter. The official baseline table does not explicitly document each channel's rate, so this choice remains an assumption. Fault files are selected from the 12k drive-end page. Both loaded channels must have equal lengths. In 99.mat the X099 channels are selected explicitly because that official file also embeds X098 channels; choosing the first matching key would silently duplicate another recording.

The dataset is not included in the repository or submission archive. A one-command optional downloader and a manual table of all file IDs are provided. Manual setup consists of downloading the indicated files, preserving their numeric filenames, placing them in a local directory, and changing DATASET_PATH. There is no synthetic-data fallback. Missing files or ambiguous channels raise an error rather than silently reducing the experiment.

5. Methodology

For each recording, the drive-end and fan-end channels are segmented at the same offsets into 2,048-point windows with stride 2,048. The two windows are concatenated in DE-then-FE order. Their 4,096 values are centered and divided by their joint standard deviation, using only that example. The vector is reshaped in row-major order into a single-channel 64 x 64 array. This is a representation of time-domain values, not a spectrogram or an FFT. No fitted normalization statistic is shared between meta-training and testing.

The paper reports more segments per class but does not give an exact stride or scaling rule. Non-overlapping windows are a conservative, reproducible choice and explicitly differ from that reported pool. At meta-test time the first half of each recording supplies support windows; the second half supplies query windows. One complete window between the halves is purged. This prevents raw sample overlap between the two roles across episodes. Windows can still be statistically correlated because they share a bearing and acquisition condition.

6. CNN Architecture

Layer	Operation	Output
Input	Paired time-domain representation	1 x 64 x 64
Block 1	Conv 3x3, 64 / BN / ReLU / MaxPool 2x2	64 x 32 x 32
Block 2	Conv 3x3, 64 / BN / ReLU / MaxPool 2x2	64 x 16 x 16
Block 3	Conv 3x3, 64 / BN / ReLU / MaxPool 2x2	64 x 8 x 8
Block 4	Conv 3x3, 64 / BN / ReLU / MaxPool 2x2	64 x 4 x 4
Classifier	Flatten 1024 / linear N-way head	N logits

The CNN has 115,011 trainable parameters for three classes and 117,061 for five classes. Learnable-rate MAML adds nine scalars for one inner step: four convolution rates, four normalization rates and one classifier rate. Convolutions use stride one and padding one. A layer's weight and bias share its rate. Padding, head layout and rate sharing are documented implementation choices. Cross-entropy receives logits and applies log-softmax internally.

7. MAML

For support loss Ls, the inner step is theta-prime = theta - alpha * grad(Ls). The outer objective is the average query loss evaluated at theta-prime; its gradient updates theta using Adam. The graph of the support gradient is retained, so the meta-update includes second-order derivatives. In the learnable variant, alpha is a layer-and-step parameter optimized by the same outer objective. Functional parameter substitution creates adapted weights without mutating the initialization.

Batch-normalization statistics are computed from support examples at the adapted weights and reused for queries. This makes evaluation non-transductive: changing other query examples cannot change a given query prediction. All CNN parameters adapt; the initialization resets for every test task. Query labels have no role in this adaptation.

8. Experimental Setup

Setting	Value
3-way split	Meta-train 1-7; meta-test 8-10
5-way split	Meta-train 1-5; meta-test 6-10
Support / query	5 support/class; 4 training queries; 15 test queries
Training data	9 fixed examples per training class, sampled per seed
Inner optimization	1 step; learned layer rates initialized at 0.01
Outer optimization	Adam; learning rate 0.001; gradient-norm cap 10
Training budget	100 iterations; 1 task per meta-batch
Evaluation	30 episodes/seed; final fixed-budget checkpoint
Random seeds	1, 2, 3; independent initialization and training pools
Execution	CPU; 2 PyTorch threads; deterministic algorithms

The learning rate of the outer optimizer, query count, normalization policy and gradient clipping are explicit implementation settings where the article is incomplete or this practical implementation adds numerical safeguards. The final iteration is always used. Test accuracy does not select a checkpoint, learning rate, seed, or training length. Training curves show adapted query performance on the meta-training pool, not on unseen test classes.

The CPU budget was selected using debug runtime before viewing the reported results. It uses 100 total training tasks per run, versus 37,500 task presentations implied by the paper's 1,500 iterations and 25-task batches. This is a major limitation, even when a resulting accuracy is high. A command in the README restores those published iteration and meta-batch settings while keeping this project's declared preprocessing and evaluation assumptions.

Before real training, both way settings passed real-data debug checks: balanced task labels, disjoint support/query sampling, disjoint class and source-file splits, finite gradients, a nonzero difference between full and first-order meta-gradients, unchanged initialization after task adaptation, and decreasing loss when optimizing a repeated training task. Debug weights are discarded. The repeated-task exercise is an optimization check, not a generalization result.

Accuracy is the fraction of correctly classified query examples. Precision, recall and F1 are macro-averaged over physical test classes after mapping episode-local labels back to global class IDs. Each episode balances query counts. Reported accuracy mean and sample standard deviation are computed across three seed-level accuracies using ddof=1. An episode-level standard deviation is saved separately and must not be substituted for seed variation. No confidence interval is claimed from dependent, repeatedly sampled windows.

Measured training time across the six runs was 12.54 minutes; final evaluation time totaled 30.59 minutes. These measurements exclude interpreter startup, downloads, debug verification, plotting and notebook rendering. Exact Python/PyTorch versions and per-run timing are stored in the metric JSON files.

9. Results

Experiment	Seed	Accuracy %	Precision %	Recall %	F1 %
3-way 5-shot	1	95.33	95.87	95.33	95.31
3-way 5-shot	2	93.85	94.72	93.85	93.80
3-way 5-shot	3	92.96	93.18	92.96	92.95
5-way 5-shot	1	58.67	61.86	58.67	59.16
5-way 5-shot	2	52.44	62.61	52.44	48.37
5-way 5-shot	3	62.98	68.55	62.98	59.26

Mean accuracy +/- sample SD: 3-way 5-shot: 94.05 +/- 1.20%; 5-way 5-shot: 58.03 +/- 5.30%. All six seeds are retained. There are 1,350 query predictions per 3-way run and 2,250 per 5-way run. The prediction CSV files allow every metric and confusion matrix to be recalculated. The optional 3-way 1-shot experiment was not run; no result for it is implied.

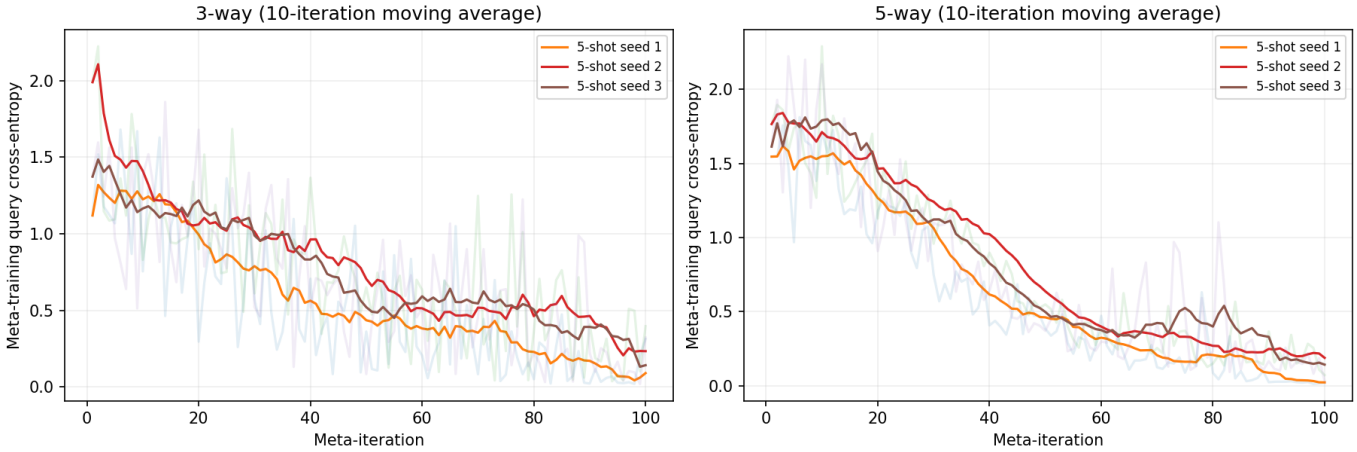


Figure 1. Meta-training query loss. Faint lines are raw values; solid lines are 10-iteration moving averages.

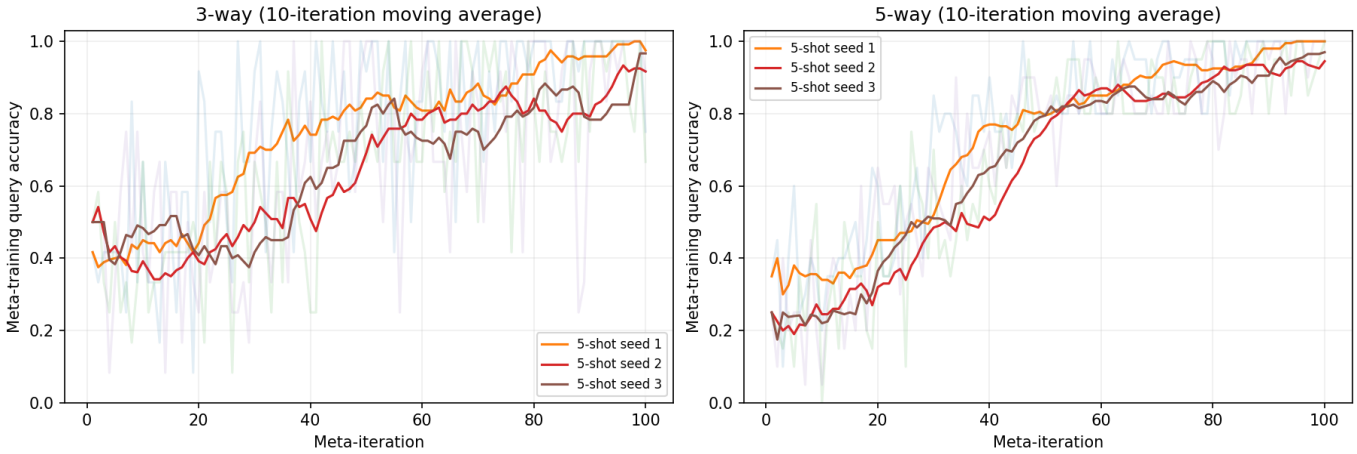


Figure 2. Adapted meta-training query accuracy. These curves are not held-out test curves.

10. Paper vs Our Results

Experiment	Paper Result	Our Result	Difference
3-way 5-shot	90.36 +/- 1.38%	94.05 +/- 1.20%	+3.69 pp
5-way 5-shot	79.98 +/- 1.17%	58.03 +/- 5.30%	-21.95 pp

Published reference values are from Table V, learnable-rate MAML [1]. The difference is ours minus the paper, in percentage points; the published results were never used as substitute measurements. A positive gap does not demonstrate superiority, and a negative gap does not isolate a failure of MAML. Training task budget, window construction, temporal support/query separation, normalization and repetition schemes are not controlled to be identical.

The article reports 1,980 segments per class while this project uses non-overlapping windows from an explicit set of recordings. Its normalization and query sampling details are incomplete. Our support-only batch statistics may also behave differently from a transductive implementation. The class label ordering and nine-example meta-training pool are matched, but matching these elements is insufficient to attribute numerical differences to one cause.

11. Confusion Matrix Analysis

The pooled 3-way matrix combines all three seeds using stable physical labels. Per-class recall is OR_007: 91.93%; OR_014: 100.00%; OR_021: 90.22%. The classes differ by outer-race defect diameter; this task evaluates separation among those three severities after adaptation from five examples each.

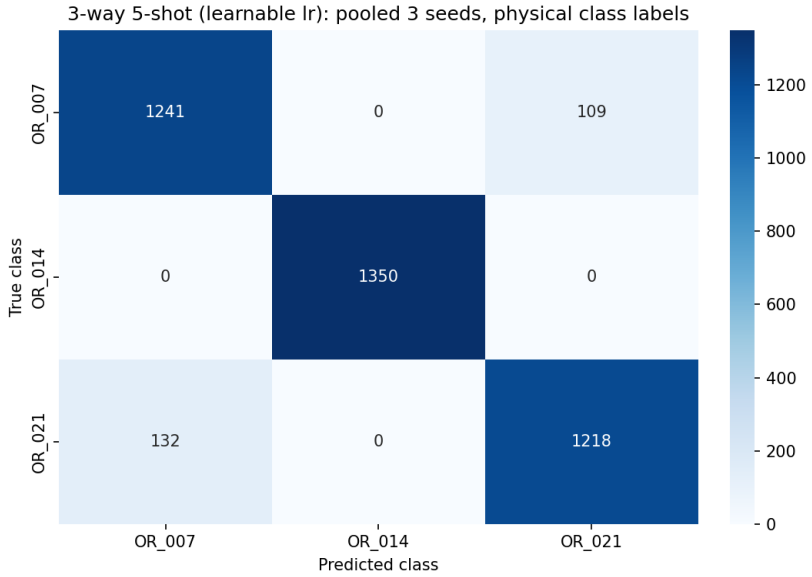


Figure 3. 3-way 5-shot confusion matrix; counts pool 4,050 query predictions across seeds.

11. Confusion Matrix Analysis (continued)

For 5-way evaluation the pooled per-class recalls are IR_014: 25.78%; IR_021: 34.52%; OR_007: 56.67%; OR_014: 95.19%; OR_021: 78.00%. The largest off-diagonal count is 812, from IR_021 to OR_021. This describes the observed errors; it does not by itself establish a physical mechanism for the confusion. The five-class experiment changes both the number of labels and their composition relative to the three-class experiment.

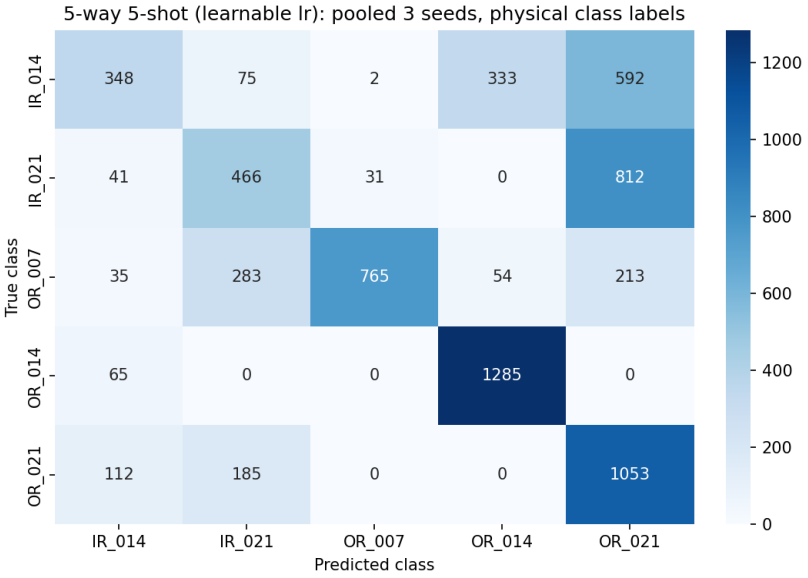


Figure 4. 5-way 5-shot confusion matrix; counts pool 6,750 query predictions across seeds.

12. Limitations

The short schedule is the principal computational limitation. Only three training seeds and 30 episodes per seed were feasible in the submission profile. The same finite windows recur across episodes; source recordings are not independent physical trials. Class-disjoint testing and non-overlapping windows prevent direct sample leakage, but do not remove dependence within a bearing or laboratory rig. The experiment does not measure generalization to a new machine, new bearing specimen, field sensor, unseen speed distribution or naturally developed damage.

There is no independent validation-class pool, so the project uses a predetermined final checkpoint and does not tune hyperparameters on test performance. Normalization, padding, head structure, healthy-signal sample rate and query policy are assumptions. We did not benchmark the fixed-rate variant, conventional supervised learning, or training from a fresh random initialization on the same test episodes. Consequently, these runs alone do not quantify the causal benefit of meta-learning or learned rates over those alternatives.

13. Conclusion

The requested practical project is operational on real CWRU recordings, with measured accuracy 3-way 5-shot: 94.05 +/- 1.20%; 5-way 5-shot: 58.03 +/- 5.30%. The project reproduces the core computational sequence: episodic sampling, support-set adaptation, query loss, full meta-gradients and an outer optimizer update. The CNN retains the article's four-block architecture, and both requested classification settings have been evaluated under independent training seeds. Saved predictions and source manifests provide traceability from raw recording to reported aggregate.

The appropriate conclusion is methodological and limited: a compact from-scratch MAML implementation can be trained and evaluated on this bearing benchmark under a practical CPU budget. Exact agreement with the original study is not established. Stronger evidence would require resolving the original window construction and normalization, matching training task exposure and repetitions, adding matched baselines, and then testing on independent bearings or an external dataset. Those are extensions rather than completed results in this submission.

To reproduce this submission, install requirements.txt, obtain the listed official MATLAB files, set DATASET_PATH, and run all notebook cells. The notebook reuses completed matching configurations by default for convenient inspection; changing RERUN_TRAINING to True reruns every seed. Reports and figures are generated from saved metrics and predictions. Raw recordings must remain outside the submitted project.

14. References

[1] Zhang, S., Ye, F., Wang, B., and Habetler, T. G. Few-Shot Bearing Fault Diagnosis Based on Model-Agnostic Meta-Learning. arXiv:2007.12851v4, 2021. https://arxiv.org/abs/2007.12851

[2] Case Western Reserve University. Bearing Data Center: Download a Data File. https://engineering.case.edu/bearingdatacenter/download-data-file

[3] Case Western Reserve University. 12k Drive End Bearing Fault Data. https://engineering.case.edu/bearingdatacenter/12k-drive-end-bearing-fault-data

[4] Case Western Reserve University. Normal Baseline Data. https://engineering.case.edu/bearingdatacenter/normal-baseline-data

[5] Finn, C., Abbeel, P., and Levine, S. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. ICML, 2017. https://proceedings.mlr.press/v70/finn17a.html

Audit files: results/metrics contains per-seed JSON, training histories, predictions and debug evidence. results/tables contains aggregate metrics, paper comparisons and split/source manifests. results/checkpoints contains the six trained model states. Environment package versions are recorded separately; the main requirements list intentionally contains only the packages needed to run the project.